Relatório de Criptoanálise — Grupo 7

Recuperação completa da mensagem confidencial

Luísa Carvalho Böhm • Murilo Costa Salem • Henrique dos Reis

25 de outubro de 2026

Resumo

Este relatório descreve, na ordem em que foram executadas, a descoberta da senha de acesso à chave pública, a decifragem AES dessa chave, a fatoração do módulo RSA, a reconstrução da chave privada, a recuperação da senha da mensagem e a decifragem do texto final. Inclui parâmetros, comandos reproduzíveis, tentativas malsucedidas e o código integral do script desenvolvido para a reconstrução RSA.

Sumário

1 Arquivos de entrada e ambiente 2

2 Descoberta da senha da chave pública 2

3 Decifragem de `key_public.en` 3

4 Fatoração do módulo e reconstrução da chave privada 3

5 Recuperação da senha da mensagem 4

6 Decifragem da mensagem final 4

7 Caminhos testados que não funcionaram 4

8 Resumo dos parâmetros encontrados 5

A Mensagem decifrada completa 5

B Código-fonte integral do script RSA desenvolvido 5

1 Arquivos de entrada e ambiente

Os quatro arquivos fornecidos foram `key_for_rsa_public.hash`, `key_public.en`, `key_for_message.en` e `message.en`, todos no diretório `7/`. O enunciado informou que a senha do primeiro arquivo começava por `pe`. Os comandos abaixo partem da raiz do projeto, em Linux, com Hashcat 7.1.2, OpenSSL 3.5.5, Python 3 e a biblioteca Python `cryptography` 46.0.5. A busca com Hashcat utilizou uma NVIDIA GeForce RTX 5090, identificada como dispositivo 1 via OpenCL.

Se o Hashcat ainda não estiver no projeto, obtenha e extraia a mesma versão antes da etapa de força bruta:

```
mkdir -p .tools
curl -fL -o .tools/hashcat-7.1.2.7z \
  https://github.com/hashcat/hashcat/releases/download/v7.1.2/hashcat-7.1.2.7z
7z x -y .tools/hashcat-7.1.2.7z -o.tools
```

Para reconstruir a chave RSA em Python, instale `cryptography` se necessário. As demais ferramentas usadas nos comandos são OpenSSL, Docker, `xxd` e `7z`.

Arquivo	Formato identificado
`key_for_rsa_public.hash`	SHA-256 em hexadecimal, 64 caracteres
`key_public.en`	Base64 de ciphertext AES, sem cabeçalho `Salted__`
`key_for_message.en`	Ciphertext RSA em hexadecimal, 48 bytes após conversão
`message.en`	Base64 do formato `openssl enc` com cabeçalho `Salted__`

2 Descoberta da senha da chave pública

O hash informado é:

```
bfc15bd9ea199751e66ddb5e92344baed2f2fe67136e12eea18281ca52ff61fb
```

O modo 1400 do Hashcat corresponde a SHA-256 bruto; `-a 3` seleciona busca por máscara e `?l` representa letras minúsculas. A tentativa inicial exigiu exatamente seis letras minúsculas após `pe`:

```
./.tools/hashcat-7.1.2/hashcat.bin -m 1400 -a 3 \
  7/key_for_rsa_public.hash 'pe?l?l?l?l?l?l' -O -w 3 -d 1 \
  --potfile-disable --restore-disable
```

Ela terminou com `Status: Exhausted`: os $26^6 = 308.915.776$ candidatos foram testados sem correspondência. A máscara foi então ampliada para dez letras minúsculas após `pe`, ou 12 caracteres no total:

```
./.tools/hashcat-7.1.2/hashcat.bin -m 1400 -a 3 \
  7/key_for_rsa_public.hash 'pe?l?l?l?l?l?l?l?l?l?l' \
  -O -w 3 -d 1 --potfile-disable --restore-disable
```

O resultado foi **`petershorxxx`**. O espaço máximo dessa segunda máscara é $26^{10} = 141.167.095.653.376$ candidatos. O Hashcat informou `Status: Cracked` após cerca de 12 segundos de ataque, com 193.476.689.920 candidatos contabilizados (0,14% do espaço). Para conferir diretamente a senha descoberta:

```
printf %s petershorxxx | openssl dgst -sha256
```

O digest deve coincidir com o conteúdo de `key_for_rsa_public.hash`.

3 Decifragem de `key_public.en`

O conteúdo Base64 desse arquivo não começa com o cabeçalho `Salted__`, de modo que a decifragem OpenSSL precisa da opção `-nosalt`. A combinação validada foi **AES-256-OFB**, codificação Base64, derivação legada de senha do OpenSSL com digest **SHA-256**, sem salt:

```
openssl enc -d -aes-256-ofb -a -nosalt -md sha256 \
  -pass pass:petershorxxx -in 7/key_public.en -out 7/key_public.pem
openssl pkey -pubin -in 7/key_public.pem -noout -text
```

O resultado é uma chave pública RSA de 382 bits, expoente $e = 65.537$, em formato PEM `BEGIN PUBLIC KEY`. O módulo decimal é:

```
6524226763420669545131292616008021977421989716443706623412713108662980402279261528264966716400244521958486921252927
```

O SHA-256 da chave pública no formato DER é:

```
dbf1517e0312436f41a2b683b296a065937e868e72a65b7a7dfc445bff32f961
```

O aviso do OpenSSL sobre derivação obsoleta é esperado: esses arquivos foram produzidos com essa derivação; usar PBKDF2 mudaria a chave gerada.

4 Fatoração do módulo e reconstrução da chave privada

A ferramenta CADO-NFS 3.0.0, com o crivo geral de corpos numéricos (GNFS), fatorou o módulo. Para reproduzir com 20 threads e estado persistido em um diretório local:

```
mkdir -p .factor-rsa382
RSA_N='6524226763420669545131292616008021977421989716443706623412 71'
RSA_N="${RSA_N}3108662980402279261528264966716400244521958486921252927"
docker run --name factor-rsa382 \
  -v "$PWD/.factor-rsa382:/work" \
  -e RSA_N="$RSA_N" \
  registry.gitlab.inria.fr/cado-nfs/cado-nfs/factoring-full:latest \
  /bin/bash -lc 'cado-nfs.py --workdir /work "$RSA_N" -t 20'
```

O tempo de parede registrado foi 935,176 segundos. Os fatores primos obtidos foram:

```
p = 22957967046861939015387584412762907411089750692880912205 67
q = 28418138026347799149269262707343375246868509913464534610 81
```

O produto pq foi comparado com o módulo da chave pública, e ambos os fatores foram verificados como primos pelo OpenSSL. A partir daí foi calculado $\lambda(n) = \mathrm{mmc}(p-1, q-1)$ e o expoente privado $d \equiv e^{-1} \pmod{\lambda(n)}$. O script integral está no Apêndice B. Para executá-lo em uma cópia limpa dos arquivos (sem as saídas pré-existentes):

```
python3 7/recover_rsa.py
openssl pkey -in 7/key_private.pem -check -noout
```

O script grava `key_private.pem` em PEM PKCS#8 sem senha e com permissão 0600. O OpenSSL retornou `Key is valid`; a chave pública extraída da privada tem o mesmo SHA-256 DER indicado acima. A chave privada é um resultado sensível: mantenha suas permissões restritas.

5 Recuperação da senha da mensagem

O arquivo `key_for_message.en` contém 96 caracteres hexadecimais, que representam um bloco RSA de 48 bytes. O script converte o hexadecimal em bytes e aplica a operação privada com padding **PKCS#1 v1.5**. A estrutura interna observada começa por `00 02`, contém bytes de preenchimento não nulos e um separador `00`. Após esse separador, o texto recuperado é:

`kristennygaa`

O valor foi salvo em `7/key_for_message.txt`, sem quebra de linha. Este comando OpenSSL reproduz a decifragem a partir da chave privada já criada:

```
tr -d '\r\n' < 7/key_for_message.en | xxd -r -p > /tmp/key_for_message.bin
openssl pkeyutl -decrypt -inkey 7/key_private.pem \
  -in /tmp/key_for_message.bin -out /tmp/key_for_message_recovered.txt
```

Por padrão, a operação RSA acima usa PKCS#1 v1.5. Compare o conteúdo do arquivo recuperado em `/tmp/` com `kristennygaa`.

6 Decifragem da mensagem final

Após conversão Base64, `message.en` começa com `Salted__`, cabeçalho do formato `openssl enc` que inclui um salt aleatório. Testes com AES e diferentes derivações produziram um único texto UTF-8 inteiramente legível: **AES-256-OFB**, Base64, salt embutido, derivação legada OpenSSL com digest **SHA-256**. O comando é:

```
openssl enc -d -aes-256-ofb -a -md sha256 \
  -pass file:7/key_for_message.txt \
  -in 7/message.en -out 7/message.dec.txt
```

O texto final contém exatamente os nomes Luísa Carvalho Böhm, Murilo Costa Salem e Henrique dos Reis, conforme exigido. O SHA-256 do arquivo UTF-8 recuperado é:

```
674d3ec5a364f98594e4edf2684680fc348e76eefa78ab8185b9b772d6148b7e
```

O Apêndice A contém a mensagem integral, com todas as linhas preservadas.

7 Caminhos testados que não funcionaram

1. **Máscara com seis letras após `pe`.** O Hashcat testou todo o espaço de 308.915.776 possibilidades e informou `Exhausted`. Falhou porque a senha real possui dez letras após `pe`; nenhuma senha de oito caracteres naquela máscara poderia produzir o hash alvo.
2. **Aplicação da chave pública ao bloco RSA.** A tentativa `openssl pkeyutl -verifyrecover -pubin` com padding PKCS#1 v1.5 retornou `invalid padding`. Usando a operação pública sem padding, o resultado foi um bloco binário sem a senha. Tecnicamente, o ciphertext deste arquivo foi recuperado pela operação com o expoente *privado* e padding PKCS#1 v1.5; a operação pública aplica o expoente errado para essa inversão.
3. **Decifragem de `key_public.en` como arquivo OpenSSL com salt.** A tentativa com `openssl enc -d -aes-256-cbc -a -pbkdf2` retornou `bad magic number`: os bytes decodificados não começam por `Salted__`. Esse erro evidencia a ausência do cabeçalho esperado; o modo AES, a derivação e o uso de salt também não coincidem com os parâmetros corretos.

8 Resumo dos parâmetros encontrados

Camada	Resultado
Senha de `key_public.en`	`petershorxxx`; SHA-256 bruto no arquivo de hash
`key_public.en`	AES-256-OFB; Base64; sem salt; KDF legado do OpenSSL com SHA-256
`key_for_message.en`	RSA de 382 bits; ciphertext hexadecimal; padding PKCS#1 v1.5
Senha recuperada para a mensagem	`kristennygaa`, 12 bytes ASCII
`message.en`	AES-256-OFB; Base64; salt no cabeçalho `Salted__`; KDF legado do OpenSSL com SHA-256
Texto final	UTF-8; reproduzido integralmente no Apêndice A

A Mensagem decifrada completa

```
MENSAGEM CONFIDENCIAL - GRUPO 7
============================================================

Destinatários autorizados: Luísa Carvalho Böhm, Murilo Costa Salem, Henrique dos Reis

Classificação: ULTRA SECRETO
Protocolo: Biuro Szyfrów - Operação Enigma 2026-2

Prezados agentes,

A operação de interceptação foi bem-sucedida. Os dados coletados
confirmam que o sistema de criptografia inimigo utilizou chaves
RSA de baixa entropia combinadas com AES-256 para proteger as
comunicações. Esta combinação, embora aparentemente segura,
possui vulnerabilidades que podem ser exploradas sistematicamente.

Nossos analistas identificaram que a chave pública RSA pode ser
fatorada em tempo viável utilizando o algoritmo GNFS (General
Number Field Sieve). A senha utilizada para proteger esta chave
foi derivada de um espaço de busca limitado, permitindo ataques
de força bruta contra o hash armazenado.

Recomendações operacionais:
  1. Utilize ferramentas especializadas para recuperação da senha.
  2. Aplique algoritmos de fatoração para a chave RSA.
  3. Documente todos os procedimentos para replicabilidade.

Este documento foi gerado exclusivamente para o grupo 7
(Luísa Carvalho Böhm, Murilo Costa Salem, Henrique dos Reis). Qualquer compartilhamento não autorizado será
detectado através da análise forense dos metadados embutidos.

Assinado digitalmente,
Biuro Szyfrów - Divisão de Criptoanálise
Data: 11/10/2026
============================================================
```

B Código-fonte integral do script RSA desenvolvido

O arquivo abaixo é o mesmo `recover_rsa.py` utilizado nas etapas 4 e 5. O projeto também contém ferramentas didáticas preexistentes, como `hashcat_python.py` e `dictionary_rules.c`; elas não foram executadas para obter os resultados apresentados neste relatório.

```
#!/usr/bin/env python3
"""Reconstrói a chave RSA do grupo 7 e recupera a chave da mensagem."""

import math
```

```
import os
from pathlib import Path

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import padding, rsa

ROOT = Path(__file__).resolve().parent
P = 22957967046861939015387584412762907411089750692880912205 67
Q = 28418138026347799149269262707343375246868509913464534610 81


def write_new_private(path: Path, data: bytes) -> None:
    """Cria um arquivo privado sem sobrescrever arquivos existentes."""
    fd = os.open(path, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    with os.fdopen(fd, "wb") as output:
        output.write(data)


def main() -> None:
    public = serialization.load_pem_public_key((ROOT / "key_public.pem").read_bytes())
    numbers = public.public_numbers()
    if not isinstance(numbers, rsa.RSAPublicNumbers) or P * Q != numbers.n:
        raise ValueError("Os fatores não correspondem à chave pública RSA")

    modulus_order = math.lcm(P - 1, Q - 1)
    d = pow(numbers.e, -1, modulus_order)
    private = rsa.RSAPrivateNumbers(
        p=P, q=Q, d=d, dmp1=d % (P - 1), dmq1=d % (Q - 1),
        iqmp=pow(Q, -1, P), public_numbers=numbers,
    ).private_key()
    if private.public_key().public_numbers() != numbers:
        raise ValueError("A chave privada reconstruída não corresponde à pública")

    ciphertext = bytes.fromhex((ROOT / "key_for_message.en").read_text().strip())
    block_size = (numbers.n.bit_length() + 7) // 8
    if len(ciphertext) != block_size or int.from_bytes(ciphertext, "big") >= numbers.n:
        raise ValueError("O bloco cifrado não corresponde ao módulo RSA")

    block = pow(int.from_bytes(ciphertext, "big"), d, numbers.n).to_bytes(block_size, "big")
    separator = block.find(b"\x00", 2)
    if block[:2] != b"\x00\x02" or separator < 10:
        raise ValueError("O bloco não contém padding PKCS#1 v1.5 válido")
    message_key = private.decrypt(ciphertext, padding.PKCS1v15())
    if message_key != block[separator + 1 :]:
        raise ValueError("A decifragem não corresponde ao bloco RSA validado")

    pem = private.private_bytes(
        serialization.Encoding.PEM,
        serialization.PrivateFormat.PKCS8,
        serialization.NoEncryption(),
    )
    private_path = ROOT / "key_private.pem"
    message_path = ROOT / "key_for_message.txt"
    if private_path.exists() or message_path.exists():
        raise FileExistsError("Um arquivo de saída já existe; nenhuma saída foi sobrescrita")
    write_new_private(private_path, pem)
    write_new_private(message_path, message_key)
    print("Chave privada validada:", private_path)
    print("Chave da mensagem recuperada:", message_key.decode("ascii"))
    print("Chave da mensagem salva em:", message_path)


if __name__ == "__main__":
    main()
```